

Daily Scholar Papers Report — 2026-07-15

2026-07-15

Daily Scholar Papers Report — 2026-07-15

[Download PDF](#)

Window covered: 2026-07-14 → 2026-07-15 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

Two Scholar-alert threads landed in the window — one followed-researcher alert (Zhenchang Xing, arXiv preprint) and one Recommended-articles batch of seven candidates. The user-curated forward queue was empty. Four candidates cleared Stage-1: an agent-trajectory watermarking preprint that formalises a *two-channel* construction robust to both deletion and rewriting; a DSN 2026 LLM-agent pipeline for binary security-patch detection from pseudocode diffs; a DSN 2026 large-scale empirical audit of batch-transfer flaws in ERC multi-token contracts; and a DSN 2026 evasion-robustness harness for code-language-model vulnerability detectors. Four candidates were dropped as saturated DL-for-smart-contract-vulnerability-detection or low-signal preprints. Preference layer remains quiet — `venue: :preprint` (posterior 0.75, n=2) is the only attribute with enough observations to weigh in; no candidate crossed the 0.30 auto-Skip or 0.65 auto-Promote thresholds.

Outstanding: 1 · **Keep:** 2 · **Borderline High-Priority:** 1

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	TRACE: A Two-Channel Robust Attribution Watermark via Complementary Embeddings for LLM-Agent Trajectories	Z. Gao, X. Li, X. Feng, J. Jiang, Y. Song, Y. Sui, Z. Xing, L. Zhu	arXiv 2607.08400 (cs.CR), Jul 2026	arXiv
2.1	SPDAgent: Leveraging LLM Agents for Context-Aware Binary Security Patch Detection via	F. Yang, Z. Gao, X. Xiao, L. Ying, B. Yin, C. Zhang, Q. Li	DSN 2026 (IEEE)	IEEE Xplore

#	Title	Authors	Venue	Link
	Pseudocode Diff Analysis			
2.2	Large-Scale Security Analysis of Multi-Token Smart Contracts: Uncovering Hidden Flaws in Batch Transfers	A. Kasthuri, S. Meisami, L. Jiang, B. Wang, Y. Duan	DSN 2026 (IEEE)	IEEE Xplore
3.1	VulLens: Enhancing Software Vulnerability Detection against Evasion Attacks	S. Sun, P. Sharma, S. Maitra, A. Stavrou, H. Wang	DSN 2026 (IEEE)	IEEE Xplore

1. Outstanding

1.1 · LLM-AGENT WATERMARKING · A two-channel agent watermark that survives an adversary with full log rewrite/delete access — selection channel is distortion-free and keyed on content (deletion-robust), tally channel is skeleton-keyed (rewrite-invariant), matches unwatermarked success rate on ALFWorld/ToolBench with detection $z \approx 100$ on long-horizon traces and >0.93 TPR@1%FPR at 70% step deletion

1.1 [TRACE: A Two-Channel Robust Attribution Watermark via Complementary Embeddings for LLM-Agent Trajectories](#) — Z. Gao, X. Li, X. Feng, J. Jiang, Y. Song, Y. Sui, Z. Xing, L. Zhu (UNSW / CSIRO Data61) — arXiv 2607.08400 (cs.CR), Jul 2026

Problem. LLM agents reach end-users through resellers that store and process the trajectory log to meter usage — and can also rewrite or delete records. When provenance is disputed, the developer must attribute a suspicious trajectory back to a specific agent using only that log, but the reseller has full read/write access to the very evidence used for attribution. Existing agent watermarks read attribution “straight off that log” and therefore break under an adversary that can rewrite content or delete steps.

Central insight — one key cannot serve both attacks. Deletion desynchronises any *position-derived* key; rewriting alters *content* and therefore any content-derived key. So a deletion-robust key must come from content, and a rewrite-robust key from position — and no single key satisfies both. TRACE’s fix: superpose *two* watermarks in complementary channels of the trajectory.

The two channels.

Selection channel — carrier is which action $b_i \in B_i$ is chosen. Context is a hash of prior actions; a DRBG keyed on that context assigns each candidate action a value $r_b \in (0, 1)$, and the agent picks b_i by a **keyed exponential race**:

$$b_i := \arg\min_{b \in B_i} \frac{1}{r_b} \{P_i[b]\} = \arg\max_{b \in B_i} r_b^{1/P_i[b]}. \quad (3)$$

The detection carrier is $\phi_i := -\ln(1 - r_{\{b_i\}})$ (Eq. 4). Pooled $X_1 := \sum \phi_i$; $z_1 := (X_1 - n)/\sqrt{n}$ (Eq. 5). Because keying is *local content*, deletion desynchronises only a memory-one window per drop — the score degrades gracefully rather than collapsing.

Tally channel — carrier is $k_i \in \{1, 2\}$, the number of records emitted for the i -th decision group. Keying uses only the log’s *skeleton* — $(\text{key}_2, \text{qid}, i)$ — never any content:

$$G_i := \begin{cases} 1 & \text{if } \rho_i < \frac{1}{2} \\ 2 & \text{else} \end{cases}, \quad \rho_i = \text{DRBG}(H(\text{key}_2), H(\text{qid} \parallel i)). \quad (6)$$

$X_2 := \sum 1[k_i \in G_i]$; $z_2 := (X_2 - n p_0)/\sqrt{(n p_0 (1-p_0))}$ (Eq. 7). Because the key touches no content, arbitrary rewriting cannot change X_2 .

Formal guarantees (verbatim theorems).

- **Theorem 5.1 (Distortion-freeness).** “Under Assumption 3.5, the rule (3) satisfies $\Pr[b_i = b] = P_i[b]$ for every $b \in B_i$.” — agent action distribution provably unchanged.
- **Theorem 5.2 (Entropy lower bound on the signal).** “For every distribution P_i , $E[\phi_i] \geq 1 + \frac{1}{2} H(P_i)$, with equality if and only if P_i is a point mass.” — each decision “pays at least half its entropy” to the watermark; deterministic decisions pay nothing.
- **Theorem 5.3 (Rewrite invariance of the tally).** “For every rewriting attack R (Definition 3.7) and every trajectory τ : $X_2(R(\tau)) = X_2(\tau)$, hence $z_2(R(\tau)) = z_2(\tau)$.”
- **Theorem 5.4 (Cost of joint erasure).** Skeleton-preserving attacks leave z_2 intact; oblivious selection-corruption forces $a \geq (h / (2+h)) m - \theta \sqrt{m}$. Erasing *both* channels forces the reseller to corrupt the trajectories it resells.

Recreated two-channel construction (Mermaid — figure embedding not permitted under arXiv non-exclusive-distrib license).

```

flowchart LR
    A[Agent step i] --> B[Context = hash of prior actions]
    B --> C[DRBG r_b in 0..1 for each b in B_i]
    C --> D[Selection channel:  
b_i = argmax r_b^(1/P_i[b])]
    D --> E1[Record action]
    D --> F[Skeleton key  
H key2, H qid concat i]
    F --> G[Tally channel: rho_i to G_i in 1 or 2]
    G --> E2[Emit k_i redundant records]
    E1 --> H[Trajectory log]
    E2 --> H
    H --> I[Detection:  
z1 from phi_i, z2 from tally]

```

Headline numbers (verbatim / Tables 2–3).

- **Detection z-scores, ALFWorld ID (principal backbone):** selection **94.15 ± 6.16**, tally **54.38 ± 2.22**. ALFWorld OOD: selection **102.53 ± 6.16**, tally **55.32 ± 1.74**. ToolBench: selection $z = 4.51$, tally $z = 5.77$. Red–Green baseline on ALFWorld ID: **37.37 ± 0.58**.
- **Success-rate parity (ALFWorld ID Avg):** Base **82.4 ± 2.1**; TRACE **83.6 ± 2.6** ($\Delta = +1.2$, statistical noise); Red–Green **74.3 ± 2.9** ($\Delta = -8.1$). ALFWorld OOD: TRACE $\Delta = -0.2$. ToolBench: TRACE $\Delta = -0.6$.

- **Deletion robustness:** “z decays gracefully from 11.34 to 4.06 at 70% deletion, still above threshold, with TPR@1%FPR never below 0.93 at any tested rate.”
- **Rewriting collapse of prior watermarks:** “RG falls from 7.09 to 0.29, AM-F from 15.85 to -4.04 with bit accuracy 0.34, and both baselines’ TPR collapses from 1.000 to 0.000 by $q = 0.8$.” TRACE tally: $z = 14.34$ at every strength.
- **Single-trajectory attribution:** TPR@1%FPR reaches 0.87–0.94 (selection) and 0.85–0.86 (tally).

Reusability.

- The **keyed exponential race** (Eq. 3) is a drop-in *distortion-free* selection sampler for any agent that samples over a discrete action set — carrying it to search agents, RL-based planners, or tool-use routers costs a single hash-and-DRBG call per decision.
- The **skeleton-only keying** trick (Eq. 6) is a clean template for any watermark that must survive an adversary with content-rewrite power: derive the key from the record’s *position and query id* alone, never from any token the adversary can edit.
- The **memory-one context window** design generalises to structured-decoding watermarks that need synchronisation-robustness against local edits.

Closing citable line: “each decision pays at least half its entropy and deterministic decisions nothing.”

2. Keep

2.1 · LLM-AGENT SECURITY PATCH DETECTION · Reframes binary silent-security-patch detection as an LLM agent loop over pseudocode diffs — the agent inspects a decompiled patched-vs-unpatched diff in context (callers, taint, control-flow) and emits per-region security labels, targeting the context gap that graph/embedding SPD classifiers cannot close

2.1 [SPDAgent: Leveraging LLM Agents for Context-Aware Binary Security Patch Detection via Pseudocode Diff Analysis](#) — F. Yang, Z. Gao, X. Xiao, L. Ying, B. Yin, C. Zhang, Q. Li — DSN 2026, 56th Annual IEEE/IFIP Int’l Conf. on Dependable Systems and Networks

Problem. Vendors routinely release *silent* security patches — no CVE assignment, no advisory — leaving downstream defenders to detect them from the shipped binary alone. Existing binary-level Security Patch Detection (SPD) methods work on assembly or graph-embedded IR and struggle with the *context* that human reviewers use: what data flows the patch touches, which caller is affected, what the pseudocode function actually does.

Method (from abstract + venue metadata). SPDAgent frames binary SPD as an LLM-agent task over *pseudocode diffs*: the decompiled version of a patched vs. unpatched binary is passed through an agent loop that inspects the diff regions in context (call sites, control-flow, taint), reasons about whether the change is security-relevant vs. functional, and emits a per-region security-patch label. The “context-aware” framing is the stated novelty: prior binary-SPD works reduce to graph or embedding classifiers over static features, whereas SPDAgent lets the LLM pull *additional* context (surrounding pseudocode, callers, variable typing) on demand.

Reusability.

- **Pseudocode-diff-as-primary-input** is transferable to any binary reasoning task where noisy assembly / IR obscures intent: silent-bug backport detection, kernel patch classification, vendor advisory reconstruction.
- The **agent-loop-over-decompiler-tools** pattern is a template for other decompilation-centric analyses (type recovery, decompilation quality assessment, malware family attribution).
- Bridges to the security-patch literature (GraphSPD, StriderSPD, “From LLMs to Agents” arXiv 2511.08060) and, combined with TRACE-style watermarking, opens a plausible chain: attribute a patch commit to a specific automated agent for provenance.

Deep-read status. IEEE Xplore only in this cycle — deep-read *deferred* pending an open-access preprint or conference PDF drop.

2.2 · SMART-CONTRACT EMPIRICAL · Large-scale empirical audit of deployed multi-token (ERC-1155) contracts, targeting batch-transfer functions specifically — a class that per-token analysers miss because the vulnerabilities live at the array boundary (recipient/sender length mismatch, per-recipient re-entrancy, silent accumulator overflow, index-vs-value confusion)

2.2 Large-Scale Security Analysis of Multi-Token Smart Contracts: Uncovering Hidden Flaws in Batch Transfers — A. Kasthuri, S. Meisami, L. Jiang, B. Wang, Y. Duan — DSN 2026, 56th Annual IEEE/IFIP Int’l Conf. on Dependable Systems and Networks

Problem. ERC token standards (ERC-20 fungible, ERC-721 NFT, ERC-1155 multi-token) provide *batch transfer* primitives so a single transaction can move multiple tokens, saving gas and boosting scalability. But batch-transfer functions cross-cut invariants that per-token analyses miss: per-recipient re-entrancy, index-vs-value confusion across the array, mismatched sender/recipient array lengths, silent overflow in accumulated amounts.

Method (from abstract). The authors collect a corpus of deployed ERC-1155 and multi-token contracts and run a taxonomy of batch-transfer-specific security checks. The framing — DSN 2026 empirical track — suggests this is a measurement paper rather than a new detector: expect quantitative prevalence data on batch-transfer vulnerabilities in production contracts.

Reusability.

- Any smart-contract analyser (Slither, Mythril, static / symbolic) can adopt the paper’s batch-transfer-specific check taxonomy as a rule pack.
- The corpus and its annotations are potentially reusable as a **benchmark** for evaluating downstream detectors on batch-transfer-specific classes.

Deep-read status. IEEE Xplore only — deep-read *deferred* pending conference PDF drop.

3. Borderline High-Priority

3.1 · EVASION-ROBUST DETECTION · Robustness harness for code-LM (CodeBERT/CodeGPT-family) vulnerability detectors, targeting the evasion-attack fragility that EaTVul (>83% attack success) exposed — the angle is a real research gap even though the underlying detectors are saturated

3.1 [VulLens: Enhancing Software Vulnerability Detection against Evasion Attacks](#) — S. Sun, P. Sharma, S. Maitra, A. Stavrou, H. Wang — DSN 2026, 56th Annual IEEE/IFIP Int’l Conf. on Dependable Systems and Networks

Problem. Pre-trained code models (CodeBERT, CodeGPT and derivatives) achieve strong benchmark accuracy on vulnerability detection but are vulnerable to evasion attacks — an adversary applies semantics-preserving code transformations (variable renaming, dead-code injection, refactoring) to flip the detector’s decision. EaTVul (USENIX Security 2024) reports >83% attack success against production DL detectors, and that fragility motivates the VulLens line.

Method (from abstract). VulLens is a robustness-oriented pipeline layered on top of code-LM-based vulnerability detectors: the abstract describes it as *enhancing* detection against evasion transformations, which typically means adversarial training over a transformation-augmented dataset, an evasion-aware fine-tuning objective, or a certification-style consistency check across semantically equivalent variants.

Reusability.

- If the transformation set and evasion-augmentation pipeline are released, they can drop into any existing code-LM detector as a plug-in robustness harness.
- The evaluation likely uses EaTVul-style attacks as its adversarial baseline — a useful triangulation point for the field.

Deep-read status. IEEE Xplore only — deep-read *deferred*; treat as a bookmark for anyone actively working on code-LM detector robustness.

Cross-Paper Synthesis

Four papers this cycle sit along a **provenance / robustness** axis. **TRACE (1.1)** carries provenance into the trace itself — a watermark that survives an adversary with full log rewrite/delete access. **SPDAgent (2.1)** carries provenance into the *patch*: given a binary, decide whether an observed diff is a silent security fix. **Multi-Token (2.2)** carries provenance into deployment — measuring which live smart contracts already carry batch-transfer bugs. **VulLens (3.1)** carries robustness into detectors themselves — hardening code-LM classifiers against evasion transformations.

The methodological arc from *behavioural watermarking* (TRACE) to *evasion-aware detection* (VulLens) is worth noting: both papers work by *superposing structure the adversary cannot cheaply remove* — TRACE via a skeleton-keyed tally channel that no content rewrite can touch; VulLens (predicted from abstract) via transformation-invariant representations that survive semantics-preserving edits. The shared design pattern — *anchor the invariant in what the adversary cannot rewrite* — is a candidate cross-cutting principle worth teasing out in a Related Work section.

Writing & Rationale Insights

TRACE’s “*deletion desynchronises a position-derived key, rewriting alters content, so a deletion-robust key must come from content and a rewrite-robust key from position, and no single key serves both*” — one sentence that motivates the entire two-channel construction. This is the template for how to frame a “why one thing can’t work, so we use two complementary things” paper. TRACE’s Theorem 5.2 pays

each decision *at least half* its entropy for the watermark, and the paper is unusually honest about that cost — it quantifies the price in closed form rather than burying it. Presentation lesson: state the cost, don't hide it. The three DSN abstracts share a pattern worth internalising: each opens with the *deployment reality* that motivates the work (silent patches, batch-transfer gas savings, evasion attacks in production) before naming the method. The framing lands harder than a method-first opener.